COMPLEJO EDUCATIVO CATOLICO SAN FRANCISCO



Nombre del proyecto: Invernadero Inteligente con Arduino

Maestro: Carlos Galdámez

Materia: Diplomado de Robótica

Alumnos: (6078903)Avilés Alisson
(5027952) García Xavier
(6652295)Hernandez Moisés
(6652086) Galdámez Melanie
(3166052) Reyes Jeremy

Grado: 2° año de bachillerato

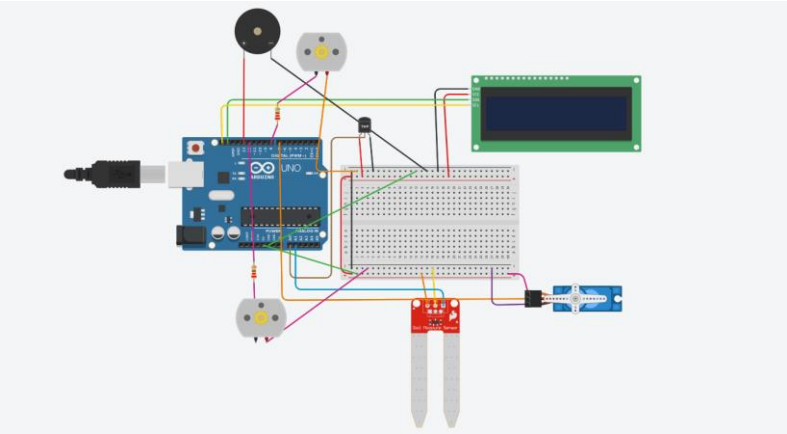
Fecha de entrega: 02/10/2026

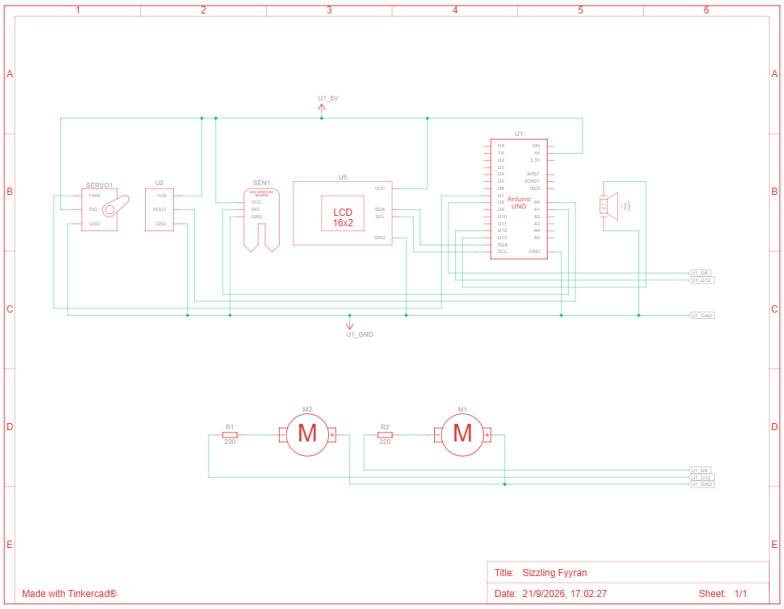
Indicé

ENSAMBLAJE Y CABLEADO 3
Plano o Diseño Mecánico 5
Organización del Cableado 5
Explicación del Proceso de Conexión de Componentes 6
Justificación de Medidas de Seguridad Eléctrica 7
DESARROLLO DE SOFTWARE / FIRMWARE 8
Imágenes del código fuente 9
INTEGRACIÓN DEL SISTEMA 10
Conexión de Sensores: 10
Conexión de Actuadores y Salidas: 10
Flujo de Funcionamiento del Sistema 11
Problemas Encontrados y Ajustes Realizados 11
SEGURIDAD DURANTE EL DESARROLLO 13
Medidas de seguridad aplicadas 13
Manejo de energía 13
Protección de componentes 13
Ejemplos de medidas de seguridad 13
REPOSITORIO EN GITHUB 14

ENSAMBLAJE Y CABLEADO

El circuito se construyó sobre una placa Arduino UNO en Tinkercad, con sensores de temperatura y humedad y con actuadores como pantalla LCD I2C, servomotor, buzzer piezométrico y actuadores de carga con relés, transistores o motores.





Name	Quantity	Component
1	1	Breadboard Small
U1	1	Arduino Uno R3
SEN1	1	Soil Moisture Sensor
U2	1	Temperature Sensor [TMP36]
M1 M2	2	DC Motor
U5	1	MCP23008-based, 32 (0x20) LCD 16 x 2 (I2C)
SERVO1	1	Positional Micro Servo
PIEZO1	1	Piezo
R1 R2	2	220 Ω Resistor

Componente	**Pin del Componente**	**Pin en Arduino / Protoboard**	**Tipo de Señal**	**Función en el Código**
Sensor TMP36	VCC / GND Vout	Bus 5V / Bus GND Pin Analógico **A0**	Entradas Analógicas	Lectura de temperatura del entorno.
Sensor Humedad Suelo	VCC / GND SIG / Out	Bus 5V / Bus GND Pin Analógico **A1**	Entradas Analógicas	Lectura de nivel de humedad.
LCD 16x2 (I2C)	VCC / GND SDA / SCL	Bus 5V / Bus GND SDA (A4) / SCL (A5)	Comunicación I2C	Mapeo visual de TEMPERATURA y HUMEDAD.
Servomotor	VCC / GND Signal	Bus 5V / Bus GND Pin Digital **7** (PWM)	Salida PWM	Apertura/cierre mecánico según la temperatura.
Buzzer	Positivo (+)	Pin Digital **13**	Salida Digital	Generación de tono (*1661 Hz*) al superar 40 °C.

	Negativo (-)	Bus GND		
Actuador Temp (Pin 8)	Control / Base Tierra	Pin Digital **8** Bus GND	Salida Digital	Activación del sistema de enfriamiento/extracción.
Actuador Riego (Pin 12)	Control / Base Tierra	Pin Digital **12** Bus GND	Salida Digital	Activación del sistema de bombeo/riego (>30% humedad).

Plano o Diseño Mecánico

El diseño físico busca distribuir la electrónica de manera eficiente y separar la zona mecánica y la zona de contacto con líquidos y humedad.

1.Ubicación de Sensores: El sensor de humedad de suelo se coloca verticalmente en el suelo para evitar lecturas falsas. El sensor de temperatura TMP36 se coloca en un lugar con aire constante y protegido del agua.

2. Ubicación de Actuadores Mecánicos y Visuales: El servomotor se coloca en la compuerta superior para ventilar el recinto cuando la temperatura supere los 30 °C. La pantalla LCD 16x2 se coloca en el panel frontal para que el operador la vea.

El buzzer y las salidas de control (pins 8 y 12) están en el compartimento superior con el Arduino para evitar problemas por humedad.

Organización del Cableado

Para evitar interferencias en las lecturas analógicas y mantener el cableado limpio, se siguieron estas directrices de ensamble:

Código de Colores de Cableado:

- **Rojo:** Línea de alimentación general (+5V).
- **Negro:** Línea de masa o tierra común (GND).
- **Verde / Amarillo:** Buses de comunicación I2C (`SDA` y `SCL`) para el LCD.
- **Naranja / Azul / Morado:** Señales analógicas de entrada (`A0`, `A1`) y salidas de control hacia el servo, buzzer y relés (`D7`, `D8`, `D12`, `D13`).
- **Canalización:** La separación entre cables de señal analógica sensible (`A0`, `A1`) y cables de conmutación de actuadores se hace con rutas paralelas independientes para evitar ruido electromagnético en lecturas analógicas.

Explicación del Proceso de Conexión de Componentes

1. Alimentación Central: Conectar salida `5V` y `GND` del Arduino UNO a rieles principal superior e inferior del protoboard.
2. Acoplamiento del Sensor de Temperatura (TMP36): Ubicar TMP36 conectando patillas de extremos a `5V` y `GND`, y terminal central de señal al pin `A0` del Arduino.
3. Módulo de Humedad de Suelo: Energizar mediante rieles protoboard y línea de señal (`SIG`) dirigir a entrada analógica `A1`.
4. Pantalla LCD con Mochila I2C: Simplificar cableado usando 4 hilos: `VCC` a 5V, `GND` a masa, `SDA` al pin A4 y `SCL` al pin A5 del Arduino.
5. Servomotor y Alarma: Servomotor recibir `5V` y `GND` directamente de líneas de potencia y cable de control conectar al pin digital `7`. Buzzer conectar entre pin digital `13` y bus de tierra.
6. Actuadores Auxiliares (Pins 8 y 12): Instalar transistores/relés de acoplamiento en salidas digitales `8` y `12`, proteger con resistencias en base para aislar cargas del puerto microcontrolador.

Justificación de Medidas de Seguridad Eléctrica

Resistencias Limitadoras e Interfaz: Instalar resistencias de protección en bases/puertas activación actuadores externos para no exceder corriente máxima por pin del ATmega328P (40 mA máximo por puerto).

Aislamiento por Zonas: Componentes expuestos a humedad (sensor de humedad de suelo) eléctrico aislar de placa microcontrolador.

Uso de Tensión Segura (SELV): Prototipo operar bajo nivel de tensión continua baja potencia (5V DC), prevenir riesgo descarga para operador durante pruebas o mantenimiento.

DESARROLLO DE SOFTWARE / FIRMWARE

El firmware del sistema está diseñado para gestionar la captura de datos físicos del entorno, el procesamiento de señales analógicas y el control directo de los componentes del proyecto. El programa se ha desarrollado en **C++** usando la plataforma Arduino, y opera en un ciclo continuo que permite la telemetría en tiempo real y la ejecución automática de acciones según las variaciones de temperatura y humedad.

La estructura del código comienza con la inclusión de dos librerías clave: **`<Adafruit_LiquidCrystal.h>`,** que controla la pantalla LCD a través de la comunicación I2C, y **`<Servo.h>`,** encargada del movimiento del servomotor. En la función `setup`, se configura la velocidad de comunicación serial a 9600 baudios, se activa la pantalla LCD con su retroiluminación, y se definen las funciones de cada terminal. Los sensores se conectan a las entradas analógicas A0 y A1, mientras que los elementos de control se asignan a las salidas digitales 7, 8, 12 y 13.

Dentro del bucle principal (`loop`), primero se lee y procesa la información proveniente de los sensores. La lectura del sensor de temperatura en A0 se convierte a grados Celsius mediante una ecuación matemática. La señal de humedad del suelo, recibida por A1, se transforma a un valor porcentual usando la función `map()` para facilitar la toma de decisiones.

Después de digitalizar los valores, el sistema actualiza la pantalla LCD mostrando los datos en ambas líneas. Al mismo tiempo, se envían los mismos valores al monitor serie para registrarlos en tiempo real. Luego, se ejecuta la lógica de actuación basada en umbrales:

- Ventilación:Si la temperatura está entre 30 °C y 40 °C, el servomotor gira a 90° para abrir la compuerta o ventana de ventilación.
- Alarma por sobretemperatura:Cuando la temperatura supera los 40 °C (hasta 50 °C), el servomotor permanece en posición abierta, el motor extractor se activa en el pin 8, y se emite una señal sonora a 1661 Hz desde el buzzer conectado al pin 13.
- Riego automático: Si la humedad del suelo es superior al 30 %, el pin 12 enciende la bomba de agua. Si no, la bomba permanece apagada.

Imágenes del código fuente

```
#include <Adafruit_LiquidCrystal.h>
#include <Servo.h>

int TEMPERATURA = 0;
int HUMEDAD = 0;

Adafruit_LiquidCrystal lcd_1(0);
Servo servo_7;

void setup()
{
  lcd_1.begin(16, 2);
  pinMode(A0, INPUT);
  pinMode(A1, INPUT);
  Serial.begin(9600);

  servo_7.attach(7, 500, 2500);
  pinMode(8, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(13, OUTPUT);

  lcd_1.setBacklight(1);
}

void loop()
{
  // Cálculo de temperatura y humedad
  TEMPERATURA = (-40 + 0.488155 * (analogRead(A0) - 20));
  HUMEDAD = map(analogRead(A1), 0, 1023, 0, 117);

  // Mostrar Temperatura en LCD y Monitor Serial
  lcd_1.setCursor(0, 0);
  lcd_1.print("T=     "); // Espacios para limpiar residuos anteriores
  lcd_1.setCursor(2, 0);
  lcd_1.print(TEMPERATURA);
  Serial.print("Temperatura: ");
  Serial.println(TEMPERATURA);

  // Mostrar Humedad en LCD y Monitor Serial
  lcd_1.setCursor(0, 1);
  lcd_1.print("H=     "); // Espacios para limpiar residuos anteriores
  lcd_1.setCursor(2, 1);
  lcd_1.print(HUMEDAD);
  Serial.print("Humedad: ");
  Serial.println(HUMEDAD);

  // Valores predeterminados (apagado / posición inicial)
  servo_7.write(0);
  digitalWrite(8, LOW);

  // Control del Servo según la temperatura
  if (TEMPERATURA > 30 && TEMPERATURA <= 40) {
    servo_7.write(90);
  }

  // Control de Alarma y Servo para temperaturas más altas
  if (TEMPERATURA > 40 && TEMPERATURA <= 50) {
    servo_7.write(90);
    digitalWrite(8, HIGH); // Encender pin 8
    tone(13, 1661, 1000);  // Tono de alarma
  }

  // Control de Humedad (Pin 12)
  if (HUMEDAD > 30) {
    digitalWrite(12, HIGH);
  } else {
    digitalWrite(12, LOW);
  }

  delay(500); // Pausa para que se alcance a leer bien en la simulación
}
```

INTEGRACIÓN DEL SISTEMA

El proceso de integración se llevó a cabo interconectando la tarjeta de desarrollo Arduino UNO con los subsistemas de monitoreo ambiental y de respuesta automatizada:

Conexión de Sensores:

- Sensor de Temperatura TMP36: Conectado al pin analógico A0 para transformar las variaciones de voltaje de salida en lecturas de temperatura en grados Celsius.
- Sensor de Humedad de Suelo: Conectado al pin analógico A1, transmitiendo una señal proporcional al contenido de humedad del sustrato.

Conexión de Actuadores y Salidas:

Servomotor (Pin 7): Integrado para el control mecánico (ej. apertura de ventilación/compuerta) mediante señales PWM (servo_7.attach(7, 500, 2500)).

Actuador de Enfriamiento/Ventilación (Pin 8): Conectado como salida digital para activar un sistema de ventilación secundario o indicador.

- Actuador de Riego/Bomba (Pin 12): Conectado a la salida digital 12 para activar el sistema de riego cuando la humedad del suelo lo requiera.
- Buzzer de Alarma (Pin 13): Conectado al pin digital 13 para emitir una alerta sonora en frecuencia de 1661 Hz cuando se detectan temperaturas críticas.
- Pantalla LCD 16x2 I2C: Conectada a las líneas de comunicación SDA (A4) y SCL (A5), permitiendo desplegar los datos en pantalla sin saturar los pines digital/analógicos del Arduino.

- Control y Lógica de Unificación: El programa principal en C++ ejecuta un bucle continuo (loop()) que toma las lecturas analógicas de los sensores, evalúa los umbrales programados y toma decisiones instantáneas sobre el servo, el riego, la ventilación y la alarma sonora.

Flujo de Funcionamiento del Sistema

El ciclo operacional del sistema sigue una secuencia ordenada de adquisición, procesamiento y respuesta:

Inicialización (setup): Se configuran la pantalla LCD I2C (lcd_1.begin(16, 2)), el monitor serial a 9600 baudios, la calibración de pulsos del servomotor y el modo de los pines digital/analógicos (A0, A1, 8, 12, 13).

1. **Lectura y Conversión de Datos:** Se adquieren las señales analógicas. El sensor de temperatura traduce la señal con la fórmula matemática especificada, y la humedad se escala de 0-1023 a un rango ajustado de 0-117 con la función map().

2. **Monitoreo Visual:** Se limpian los residuos anteriores en el LCD e imprimen los valores actualizados de TEMPERATURA y HUMEDAD tanto en la pantalla como en el puerto Serial.
3. **Ejecución de Reglas de Control:**

    - Si la temperatura supera los 30°C, el servo rota a 90°.
    - Si supera los 40°C, además de abrir el servo, se enciende la salida digital 8 y se activa la alarma piezoeléctrica a 1661 Hz por 1 segundo.
    - Si la humedad es mayor al 30%, el pin 12 se coloca en estado alto (HIGH) para activar el sistema de riego/bomba.
4. **Retroalimentación:** Un retardo de 500 ms garantiza estabilidad visual en el LCD antes de iniciar el siguiente ciclo de medición.

Problemas Encontrados y Ajustes Realizados

Durante la fase de integración y simulación en Tinkercad se presentaron los siguientes inconvenientes técnicos:

- **Inestabilidad y sobreposición de caracteres en la pantalla LCD:**
    - Problema: Al cambiar rápidamente entre valores numéricos (por ejemplo, pasar de 100 a 9 °C), quedaban dígitos "fantasma" o residuos impresos en la pantalla LCD.
    - Ajuste: Se añadieron espacios en blanco intencionales antes de imprimir los datos (lcd_1.print("T= ") y lcd_1.print("H= ")) para sobrescribir y limpiar los residuos en cada ciclo de actualización.

- **Caídas de tensión y carga en la tarjeta Arduino por el Servomotor y Buzzer:**
    - Problema: La demanda simultánea de corriente del servomotor y el buzzer al activarse la alerta sonora superaba el límite del regulador del Arduino UNO en simulación.
    - Ajuste: Se independizó la línea de potencia del servomotor conectándolo a los rieles del protoboard y manteniendo la referencia a tierra (GND) unificada con el Arduino.
- **Desfasamiento en el tiempo de respuesta debido a retardos en la alarma:**
    - Problema: El uso de la función tone() con tiempo definido de 1000 ms bloqueaba brevemente la actualización regular del monitoreo cuando la temperatura alcanzaba rangos de emergencia.
    - Ajuste: Se ajustó la pausa general del bucle a delay(500) para compensar los tiempos de ejecución y mantener una actualización constante en el Monitor Serial sin congelar el sistema.

SEGURIDAD DURANTE EL DESARROLLO

Medidas de seguridad aplicadas

Durante el desarrollo del invernadero inteligente se tomaron diferentes medidas de seguridad para evitar accidentes y daños en los componentes electrónicos. Se revisaron las conexiones antes de encender el sistema y se evitó manipular los circuitos mientras estaban conectados a la fuente de energía.

También se mantuvieron organizados y protegidos los cables para evitar cortocircuitos o desconexiones. Debido a que el proyecto utiliza sensores y componentes electrónicos cerca de las plantas y el agua, se procuró mantener los circuitos alejados de la humedad.

Manejo de energía

Se utilizaron las fuentes de alimentación adecuadas para cada componente, verificando que el voltaje fuera compatible antes de realizar las conexiones. También se desconectó la alimentación antes de modificar o revisar el circuito.

Para evitar daños, se tuvo cuidado de no conectar los componentes de forma incorrecta y se revisaron las conexiones antes de poner en funcionamiento el sistema.

Protección de componentes

Los sensores, cables y demás componentes electrónicos se colocaron de manera que estuvieran protegidos del agua y la humedad del invernadero. También se evitó ejercer fuerza sobre los cables o conexiones.

En caso de detectar algún componente caliente, una conexión suelta o un funcionamiento extraño, se debía desconectar el sistema y revisar el problema antes de continuar.

Ejemplos de medidas de seguridad

- Revisar las conexiones antes de encender el sistema.
- Evitar que el agua entre en contacto con los componentes electrónicos.
- Utilizar el voltaje adecuado para cada componente.
- Mantener los cables ordenados y protegidos.
- Proteger los sensores de la humedad excesiva.
- Manipular los componentes con cuidado para evitar dañarlos.

REPOSITORIO EN GITHUB

https://github.com/jeredanrey2904-stack/invernadero-inteigente-/blob/main/README.md

Este proyecto integra un sistema automatizado de control ambiental programado en Arduino que monitorea en tiempo real la temperatura y la humedad mediante sensores analógicos en los pines A0 y A1, desplegando los datos en un monitor serial y en una pantalla LCD 16x2. A través de su lógica de control, el código reacciona a los valores medidos girando un servomotor a 90° en el pin 7 para abrir una compuerta al superar los 30 °C, encendiendo un motor extractor en el pin 8 junto a una alarma en el pin 13 si la temperatura excede los 40 °C, y activando la bomba de riego en el pin 12 cuando la humedad supera el 30 %, todo respaldado por una pausa de 500 ms que garantiza la estabilidad de las lecturas.

```
#include <Adafruit_LiquidCrystal.h>
#include <Servo.h>

int TEMPERATURA = 0;
int HUMEDAD = 0;

Adafruit_LiquidCrystal lcd_1(0);
Servo servo_7;

void setup()
{
  lcd_1.begin(16, 2);
  pinMode(A0, INPUT);
  pinMode(A1, INPUT);
  Serial.begin(9600);

  servo_7.attach(7, 500, 2500);
  pinMode(8, OUTPUT);
  pinMode(12, OUTPUT);
  pinMode(13, OUTPUT);

  lcd_1.setBacklight(1);
}

void loop()
{
  // Cálculo de temperatura y humedad
  TEMPERATURA = (-40 + 0.488155 * (analogRead(A0) - 20));
  HUMEDAD = map(analogRead(A1), 0, 1023, 0, 117);

  // Mostrar Temperatura en LCD y Monitor Serial
  lcd_1.setCursor(0, 0);
  lcd_1.print("T=     "); // Espacios para limpiar residuos anteriores
  lcd_1.setCursor(2, 0);
  lcd_1.print(TEMPERATURA);
  Serial.print("Temperatura: ");
  Serial.println(TEMPERATURA);

  // Mostrar Humedad en LCD y Monitor Serial
  lcd_1.setCursor(0, 1);
  lcd_1.print("H=     "); // Espacios para limpiar residuos anteriores
  lcd_1.setCursor(2, 1);
  lcd_1.print(HUMEDAD);
  Serial.print("Humedad: ");
```

```
  Serial.println(HUMEDAD);

  // Valores predeterminados (apagado / posición inicial)
  servo_7.write(0);
  digitalWrite(8, LOW);

  // Control del Servo según la temperatura
  if (TEMPERATURA > 30 && TEMPERATURA <= 40) {
    servo_7.write(90);
  }

  // Control de Alarma y Servo para temperaturas más altas
  if (TEMPERATURA > 40 && TEMPERATURA <= 50) {
    servo_7.write(90);
    digitalWrite(8, HIGH); // Encender pin 8
    tone(13, 1661, 1000);  // Tono de alarma
  }

  // Control de Humedad (Pin 12)
  if (HUMEDAD > 30) {
    digitalWrite(12, HIGH);
  } else {
    digitalWrite(12, LOW);
  }

  delay(500); // Pausa para que se alcance a leer bien en la simulación
}
```